Lab Sheet 1

Experiment Name : Real world data collection, cleaning and preprocessing using python

Steps

1.
Download a real-world dataset such as the Titanic Survival Dataset or IBM HR Employee Attrition Dataset from Kaggle.

2.
Import the required libraries (Pandas, NumPy, Matplotlib, and Seaborn).

3.
Load the dataset into a Pandas DataFrame and inspect its structure using head(), info(), and describe().

4.
Identify missing values and handle them using appropriate techniques such as mean, median, mode, or row/column removal.

5.
Detects and removes duplicate records to ensure data consistency.

6.
Standardize inconsistent categorical values and convert them into numerical format using Label Encoding or One-Hot Encoding.

Solution ->

STEP 1 : IMPORT LIBRARIES

```
[3]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2 : LOAD THE DATASET

```
[4]
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

```
Dataset loaded successfully!
Shape: (891, 12)
```

STEP 3 : INSPECT THE DATASET

+ Code + Text

```
[5]
print("First 5 rows:")
display(df.head())
print("\nDataset Information:")
df.info()
print("\nStatistical Description:")
display(df.describe())
```

First 5 rows:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Statistical Description:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

STEP 4 : IDENTIFY AND HANDLE MISSING VALUES

```
[7]
print("Missing values before cleaning:")
print(df.isnull().sum())
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df.drop("Cabin", axis=1, inplace=True)
print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values before cleaning:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
Missing values after cleaning:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

STEP 5 : DETECT AND REMOVE DUPLICATE RECORDS

```
print("Number of duplicate rows:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Shape after removing duplicates:", df.shape)
```

```
Number of duplicate rows: 0
Shape after removing duplicates: (891, 11)
```

STEP 6 : STANDARDIZE AND ENCODE CATEGORICAL VALUES

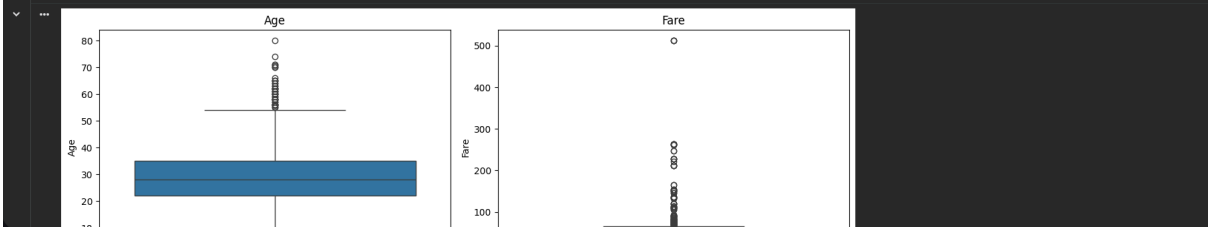
```
print("Categorical columns:")
print(df.select_dtypes(include=["object"]).columns)
df["Sex"] = df["Sex"].str.lower().str.strip()
df["Embarked"] = df["Embarked"].str.upper().str.strip()
df = pd.get_dummies(
    df,
    columns=["Sex", "Embarked"],
    drop_first=True
)
print("Dataset after encoding:")
display(df.head())
```

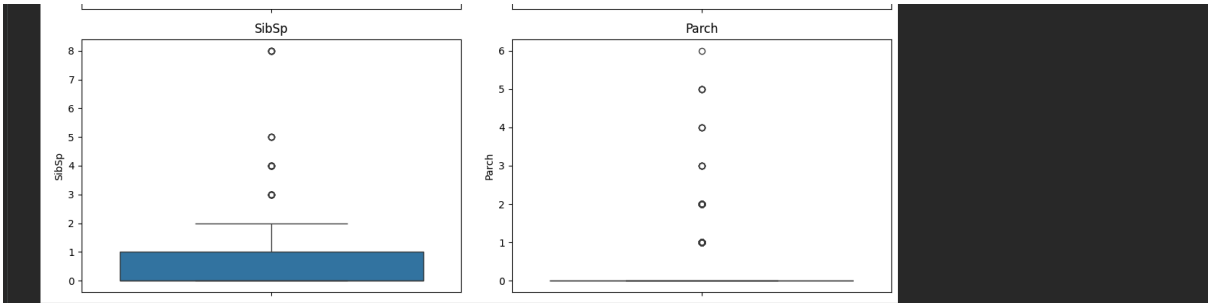
```
Categorical columns:
Index(['Name', 'Sex', 'Ticket', 'Embarked'], dtype='object')
Dataset after encoding:
```

	PassengerId	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S
0	1	0	3	Braund, Mr. Owen Harris	22.0	1	0	A/5 21171	7.2500	True	False	True
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0	PC 17599	71.2833	False	False	False
2	3	1	3	Heikkinen, Miss. Laina	26.0	0	0	STON/O2. 3101282	7.9250	False	False	True
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0	113803	53.1000	False	False	True
4	5	0	3	Allen, Mr. William Henry	35.0	0	0	373450	8.0500	True	False	True

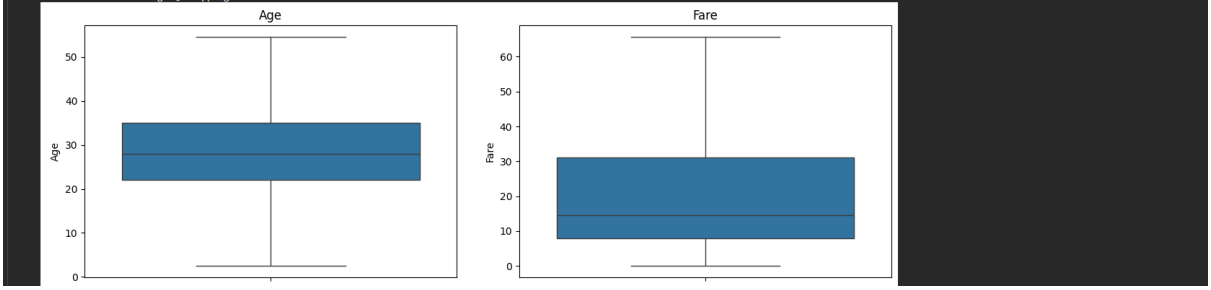
STEP 7 : DETECT AND TREAT OUTLIERS USING IQR

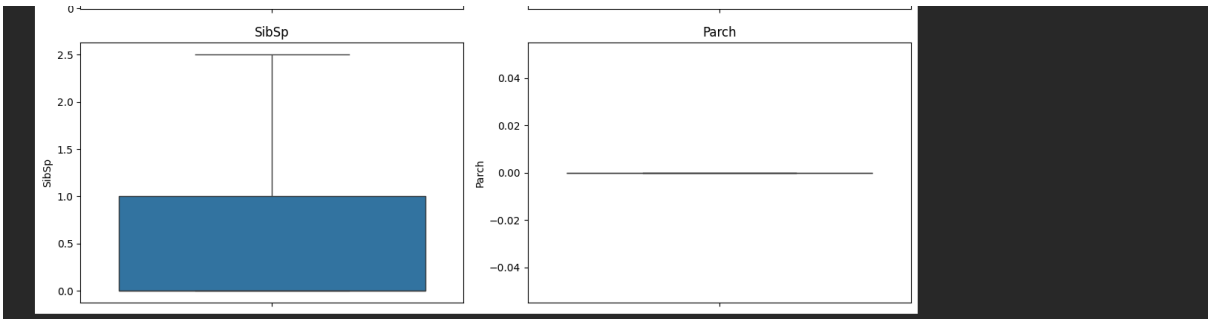
```
numerical_columns = ["Age", "Fare", "SibSp", "Parch"]
plt.figure(figsize=(12, 8))
for i, column in enumerate(numerical_columns, 1):
    plt.subplot(2, 2, i)
    sns.boxplot(y=df[column])
    plt.title(column)
plt.tight_layout()
plt.show()
for column in numerical_columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df[column] = df[column].clip(
        lower_bound,
        upper_bound
    )
print("Outliers treated using IQR capping.")
plt.figure(figsize=(12, 8))
for i, column in enumerate(numerical_columns, 1):
    plt.subplot(2, 2, i)
    sns.boxplot(y=df[column])
    plt.title(column)
plt.tight_layout()
plt.show()
```






```
Outliers treated using IQR capping.
```





STEP 8 : NORMALIZE NUMERICAL ATTRIBUTES

```
from sklearn.preprocessing import StandardScaler
scale_columns = ["Age", "Fare", "SibSp", "Parch"]
scaler = StandardScaler()
df[scale_columns] = scaler.fit_transform(df[scale_columns])
print("Numerical features standardized.")
display(df.head())
```

```
Numerical features standardized.
```

	PassengerId	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S
0	1	0	3	Braund, Mr. Owen Harris	-0.583432	0.810220	0.0	A/5 21171	-0.820552	True	False	True
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0.742685	0.810220	0.0	PC 17599	2.031623	False	False	False
2	3	1	3	Heikkinen, Miss. Laina	-0.251903	-0.602512	0.0	STON/O2. 3101282	-0.787578	False	False	True
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.494038	0.810220	0.0	113803	1.419297	False	False	True
4	5	0	3	Allen, Mr. William Henry	0.494038	-0.602512	0.0	373450	-0.781471	True	False	True

STEP 9 : FEATURE ENGINEERING

```
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[-np.inf, -0.5, 0.5, 1.5, np.inf],
    labels=["Young", "Adult", "Middle_Age", "Senior"]
)
print("New features created:")
display(df[["FamilySize", "AgeGroup"]].head())
df = pd.read_csv(url)
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df.drop("Cabin", axis=1, inplace=True)
df.drop_duplicates(inplace=True)
df["Sex"] = df["Sex"].str.lower().str.strip()
df["Embarked"] = df["Embarked"].str.upper().str.strip()
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0, 12, 18, 35, 60, np.inf],
    labels=["Child", "Teenager", "Adult", "Middle_Age", "Senior"]
)
```

```
)
df = pd.get_dummies(
    df,
    columns=["Sex", "Embarked", "AgeGroup"],
    drop_first=True
)
numerical_columns = ["Age", "Fare", "SibSp", "Parch", "FamilySize"]
for column in numerical_columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df[column] = df[column].clip(
        lower_bound,
        upper_bound
    )
scale_columns = ["Age", "Fare", "SibSp", "Parch", "FamilySize"]
scaler = StandardScaler()
df[scale_columns] = scaler.fit_transform(df[scale_columns])
print("Feature engineering and preprocessing completed.")
display(df.head())
```

```
New features created:
```

	FamilySize	AgeGroup
0	1.810220	Young
1	1.810220	Middle_Age
2	0.397488	Adult
3	1.810220	Adult
4	0.397488	Adult

```
Feature engineering and preprocessing completed.
```

	PassengerId	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	FamilySize	Sex_male	Embarked_Q	Embarked_S	AgeGroup_Teenager	AgeGroup_Adult	AgeGroup_Middle_Age	AgeGroup_Senior
0	1	0	3	Braund, Mr. Owen Harris	-0.583432	0.810220	0.0	A/5 21171	-0.820552	0.366685	True	False	True	False	True	False	False
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0.742685	0.810220	0.0	PC 17599	2.031623	0.366685	False	False	False	False	False	True	False
2	3	1	3	Heikkinen, Miss. Laina	-0.251903	-0.602512	0.0	STON/O2. 3101282	-0.787578	-0.727841	False	False	True	False	True	False	False
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.494038	0.810220	0.0	113803	1.419297	0.366685	False	False	True	False	True	False	False

STEP 10 : SAVE THE CLEANED DATASET

```
output_file = "titanic_cleaned_preprocessed.csv"
df.to_csv(output_file, index=False)
print("File saved successfully!")
print("Filename:", output_file)
print("Final dataset shape:", df.shape)
from google.colab import files
files.download("titanic_cleaned_preprocessed.csv")
```

```
File saved successfully!
Filename: titanic_cleaned_preprocessed.csv
Final dataset shape: (891, 17)
```

Questions

1.
Why is data preprocessing considered one of the most important phases in data analytics?
2.
Explain different methods of handling missing values with suitable examples.
3.
Differentiate between Label Encoding and One-Hot Encoding.
4.
What are outliers? How can they affect analytical results?
5.
Explain the difference between normalization and standardization.
6.
Why should duplicate records be removed before analysis?
7.
What is feature engineering? Give two practical examples.
8.
Which preprocessing techniques would you apply to the IBM HR Employee Attrition dataset and why?
9.
How does poor-quality data affect machine learning model performance?
10.
Name any three Python libraries commonly used for data preprocessing.

Answers: **1.**Data preprocessing is important because real-world data is often **incomplete, inconsistent, noisy, or duplicated**. Preprocessing improves the quality of data before analysis or machine learning.

It includes handling missing values, removing duplicates, treating outliers, encoding categorical data, and scaling numerical features. Good preprocessing helps produce **more accurate, reliable, and meaningful results**.

2.Common methods for handling missing values are:

1. **Mean:** Replace missing numerical values with the average.
    - Example: Missing `Age` → replace with mean age.
2. **Median:** Replace missing numerical values with the middle value. It is useful when data contains outliers.
    - Example: Missing `Salary` → replace with median salary.
3. **Mode:** Replace missing categorical values with the most frequently occurring value.
    - Example: Missing `Department` → replace with the most common department.
4. **Row removal:** Remove records containing missing values when only a few records are affected.
    - Example: Delete rows where important information such as `Customer_ID` is missing.
5. **Column removal:** Remove a column when a very large percentage of its values are missing.
    - Example: Remove a column where 80–90% of values are missing.

3.

Label Encoding	One-Hot Encoding
Assigns a numerical value to each category.	Creates a separate binary column for each category.
Example: Male = 0, Female = 1.	Male = `[1,0]`, Female = `[0,1]`.
Uses fewer columns.	Can create many columns.
May imply an order between categories.	Does not imply any order.
Suitable for ordinal data such as Low, Medium, High.	Suitable for nominal data such as City or Department.

---

4. Outliers are data points that are significantly different from the majority of observations.

For example, if most employee salaries are between ₹20,000 and ₹80,000 but one employee has a salary of ₹10,00,000, it may be an outlier.

Outliers can:

- Distort the mean and standard deviation.
- Affect statistical analysis.
- Influence regression models.
- Reduce machine learning model performance.
- Produce misleading conclusions.

Outliers can be detected using **box plots, IQR, or Z-score** and can be removed or capped when appropriate.

5.

Normalization	Standardization
Usually scales data to 0–1.	Centers data around 0.
Uses minimum and maximum values.	Uses mean and standard deviation.
Useful for bounded features and some distance-based algorithms.	Commonly used for algorithms such as SVM, logistic regression, and PCA.

6. Duplicate records represent the **same observation more than once**. If they are not removed, they can give certain observations more importance than they deserve.

Duplicates can:

- Produce incorrect statistics.
- Bias analysis.
- Affect averages and counts.
- Increase dataset size unnecessarily.
- Reduce the reliability of machine learning models.

Therefore, duplicate records should be identified and removed before analysis.

7. **Feature engineering** is the process of creating new useful features from existing data to improve analysis or machine learning performance.

Example 1 — Family Size:

For the Titanic dataset:

FamilySize = SibSp + Parch + 1

This represents the total number of people in a passenger's family.

Example 2 — Age Group:

Instead of using exact age, passengers can be grouped as:

0–12 → Child
13–18 → Teenager
19–35 → Adult
36–60 → Middle Age
60+ → Senior

These new features can make patterns easier for a model to identify.

8. For the **IBM HR Employee Attrition dataset**, I would apply the following techniques:

1. **Missing value handling** – Check for missing values and replace them using mean, median, or mode where appropriate.

2. **Duplicate removal** – Remove duplicate employee records.
3. **Categorical encoding** – Convert attributes such as `Gender`, `Department`, `JobRole`, and `BusinessTravel` into numerical values using One-Hot or Label Encoding.
4. **Outlier treatment** – Detects extreme values in features such as `MonthlyIncome`, `Age`, and `TotalWorkingYears`.
5. **Feature scaling** – Standardize numerical features when required by the machine learning algorithm.
6. **Feature engineering** – Create useful features such as income categories or years-at-company groups.
7. **Remove irrelevant features** – Remove columns that do not contribute useful information, such as employee identifiers.

These techniques make the dataset **cleaner, consistent, and suitable for machine learning**.

9. Poor-quality data can significantly reduce machine learning performance. Missing values, incorrect data, duplicates, inconsistent categories, and outliers can cause the model to learn **incorrect patterns**.

This can lead to:

- Lower accuracy.
- Incorrect predictions.
- Overfitting or underfitting.
- Biased results.
- Poor generalization to new data.

Therefore, **high-quality and properly preprocessed data is essential for building reliable machine learning models**.

10. Three commonly used Python libraries are:

1. **Pandas** – Used for loading, cleaning, filtering, and manipulating datasets.
2. **NumPy** – Used for numerical operations and handling arrays.
3. **Scikit-learn (`sklearn`)** – Provides preprocessing tools such as `StandardScaler`, `MinMaxScaler`, `LabelEncoder`, and `OneHotEncoder`.